Лабораторная работа 4

Счётчик калорий

1 Диаграмма пакетов

Диаграмма пакетов показывает, на какие логические части разбита программа «Счётчик калорий» и как эти части зависят друг от друга. Диаграммы лабораторной работы 4 построены для целевой системы: к программе лабораторной работы 2 добавлен поиск продукта по штрихкоду во внешней базе продуктов, который появился на диаграмме вариантов использования в лабораторной работе 3. Имена пакетов, компонентов и классов записаны на английском языке, как в коде.

Программа разбита на пять пакетов. Пакет UI — интерфейс пользователя: класс Program с консольным меню. Пакет Accounting — учёт порций: обработчик учёта порции PortionHandler. Пакет Data — каталог продуктов ProductCatalog и дневник питания FoodDiary, которые хранят продукты и записанные порции. Пакет Integration — клиент внешней базы продуктов ProductDatabaseClient, который по штрихкоду получает название и калорийность продукта. Пакет Model — классы предметной области: абстрактный класс еды Food, продукт Product, приготовленная партия CookedBatch, ингредиент Ingredient и порция Portion. Содержимое пакетов показано внутри них прямоугольниками классов, как в примере пособия.

Пунктирные стрелки — зависимости «use»: хотя бы один класс пакета использует класс пакета, на который указывает стрелка. Интерфейс пользователя вызывает операции учёта порций, а продукт добавляет в каталог и калории за день получает из дневника напрямую, поэтому пакет UI зависит от Accounting и Data. Обработчик учёта порции хранит ссылки на каталог и дневник, поэтому Accounting зависит от Data. Каталог, не найдя продукт по штрихкоду, обращается к клиенту внешней базы, поэтому Data зависит от Integration. Продукты, партии и порции передаются между всеми частями программы, поэтому каждый пакет зависит от Model. Причина каждой зависимости приведена в таблице 2.

Взаимных и циклических зависимостей на диаграмме нет. Все стрелки направлены в одну сторону — от интерфейса к модели: UI, Accounting, Data, Integration, Model, — и ни одна не идёт обратно. Пакет Model ни от чего не зависит: классы предметной области не знают, кто их хранит и показывает. Это следует из направлений ассоциаций на диаграмме классов лабораторной работы 2: дневник знает свои порции, а порция дневник не знает; каталог знает продукты, а продукт каталог не знает.

Цикл появился бы, если бы каталог и дневник лежали в пакете Model рядом с продуктами и порциями. Каталог обращается к клиенту внешней базы, а клиент создаёт объекты Product, и пакеты Model и Integration зависели бы друг от друга. Поэтому каталог и дневник вынесены в отдельный пакет Data.

В языке C# пакетам соответствуют пространства имён. В коде лабораторной работы 2 все классы пока лежат в одном пространстве имён CalorieCounter, а класса ProductDatabaseClient ещё нет. При реализации поиска по штрихкоду каждому пакету будет соответствовать своё пространство имён, например CalorieCounter.Data. Диаграмма представлена на рисунке 1, её описание — в таблице 1.

Таблица 1 – Описание диаграммы пакетов

Поле	**Описание**
Вопрос	Из каких логических частей состоит программа и в каком направлении они зависят друг от друга?
Читатель	Разработчик
Что сделает читатель	Разложит классы по пространствам имён и, добавляя новый класс, проверит, что его зависимости не создают цикла между пакетами
Что показано	– пять пакетов программы и классы каждого пакета; – зависимости «use» между пакетами, направленные от интерфейса к модели; – пакет Integration для поиска продукта по штрихкоду во внешней базе; – отсутствие взаимных и циклических зависимостей.
Что не показано	– связи между классами внутри пакетов — они показаны на диаграмме классов лабораторной работы 2; – операции классов и то, какие именно классы другого пакета использует каждый класс; – устройство внешней базы продуктов.
Допущения	– диаграмма описывает целевую систему: в коде лабораторной работы 2 все классы лежат в одном пространстве имён, а класса ProductDatabaseClient ещё нет; – зависимость между пакетами показана, если её требует хотя бы один класс пакета; – класс Program на диаграмме классов не показан, здесь он представляет весь интерфейс пользователя.

Таблица 2 – Зависимости между пакетами

Зависимость	**Чем обусловлена**
UI → Accounting	Program вызывает операции CreateBatch, Preview и Confirm обработчика учёта порции
UI → Data	Program добавляет продукт через ProductCatalog.AddProduct и получает калории за день через FoodDiary.CalculateDayKcal
UI → Model	Program создаёт продукты Product и показывает еду Food, приготовленные партии CookedBatch и порции Portion
Accounting → Data	PortionHandler хранит ссылки на каталог и дневник, ищет продукты через FindProduct и записывает порции через AddPortion
Accounting → Model	PortionHandler создаёт партии CookedBatch и порции Portion из еды Food
Data → Integration	ProductCatalog обращается к ProductDatabaseClient, если продукта с таким штрихкодом нет в каталоге
Data → Model	ProductCatalog хранит продукты Product, FoodDiary хранит порции Portion
Integration → Model	ProductDatabaseClient создаёт продукт Product по ответу внешней базы
Model	ни от какого пакета не зависит

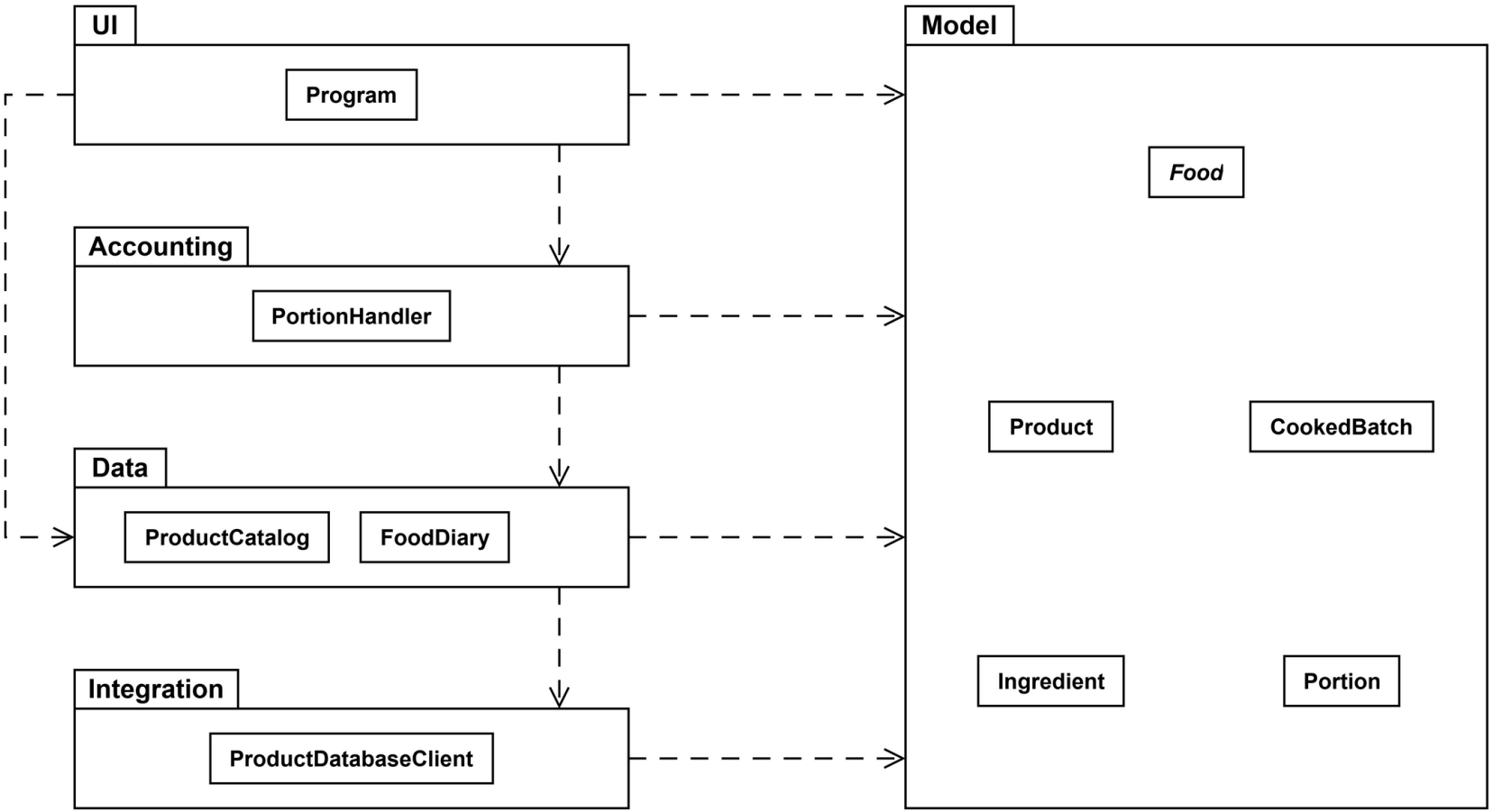


Рисунок 1 – Архитектура счётчика калорий

2 Диаграмма компонентов

Диаграмма компонентов представляет программу как набор заменяемых модулей — компонентов, которые связаны друг с другом только через интерфейсы. Компонент можно заменить другим, если новый предоставляет и требует те же интерфейсы. Диаграмма построена для той же целевой системы, что и диаграмма пакетов.

Вся программа показана составным компонентом CalorieCounter, его внутренняя структура — в отдельном разделе «internal structure», как в примере пособия. Внутри пять компонентов: консольный интерфейс ConsoleUI (класс Program), учёт порций PortionAccounting (PortionHandler), каталог продуктов ProductCatalog, дневник питания FoodDiary и клиент внешней базы продуктов ProductDatabaseClient. Вне программы находится внешний компонент ProductDatabase — внешняя база продуктов. Классы модели Food, Product, CookedBatch, Ingredient и Portion отдельным компонентом не показаны: это данные, которые компоненты передают друг другу через интерфейсы.

Предоставляемый интерфейс изображён кружком на линии от компонента, который его реализует, требуемый — полукругом на линии от компонента, который его использует. Кружок в полукруге означает, что требуемый интерфейс одного компонента обеспечен предоставляемым интерфейсом другого. Каждый интерфейс назван по операциям, которые в нём собраны; операции перечислены в таблице 4.

Каталог и дневник предоставляют по два интерфейса, потому что интерфейсу пользователя и учёту порций нужны разные операции. Интерфейс пользователя добавляет продукты (ICatalogEditing) и узнаёт калории за день (IDiaryReport), а учёт порций ищет продукт по названию при приготовлении партии (IProductSearch) и записывает подтверждённую порцию (IDiaryRecording). Так каждый компонент зависит только от тех операций, которые вызывает.

Клиент ProductDatabaseClient — единственный компонент, который обращается к внешней базе продуктов. Каталог требует от него интерфейс IBarcodeLookup — найти продукт по штрихкоду — и не знает, как устроена внешняя база. Клиент требует интерфейс внешней системы IProductApi через порт — квадрат на границе программы. Порт показывает, что запросы к внешней базе выходят за пределы программы только в этой точке. Если сменить внешнюю базу, заменить придётся только клиент: интерфейс IBarcodeLookup для каталога останется прежним.

Циклических зависимостей между компонентами нет. Требования направлены в одну сторону: ConsoleUI требует интерфейсы учёта порций, каталога и дневника, PortionAccounting — каталога и дневника, ProductCatalog — клиента внешней базы,

клиент — внешней базы. FoodDiary и ProductDatabase ничего не требуют. Ни один компонент не требует интерфейса компонента, который сам, напрямую или через другие, требует его интерфейс.

Интерфейсы на диаграмме проектные: в коде лабораторной работы 2 их нет, и компоненты обращаются к классам напрямую. Операции интерфейсов — это открытые операции соответствующих классов. Диаграмма представлена на рисунке 2, её описание — в таблице 3, интерфейсы — в таблице 4.

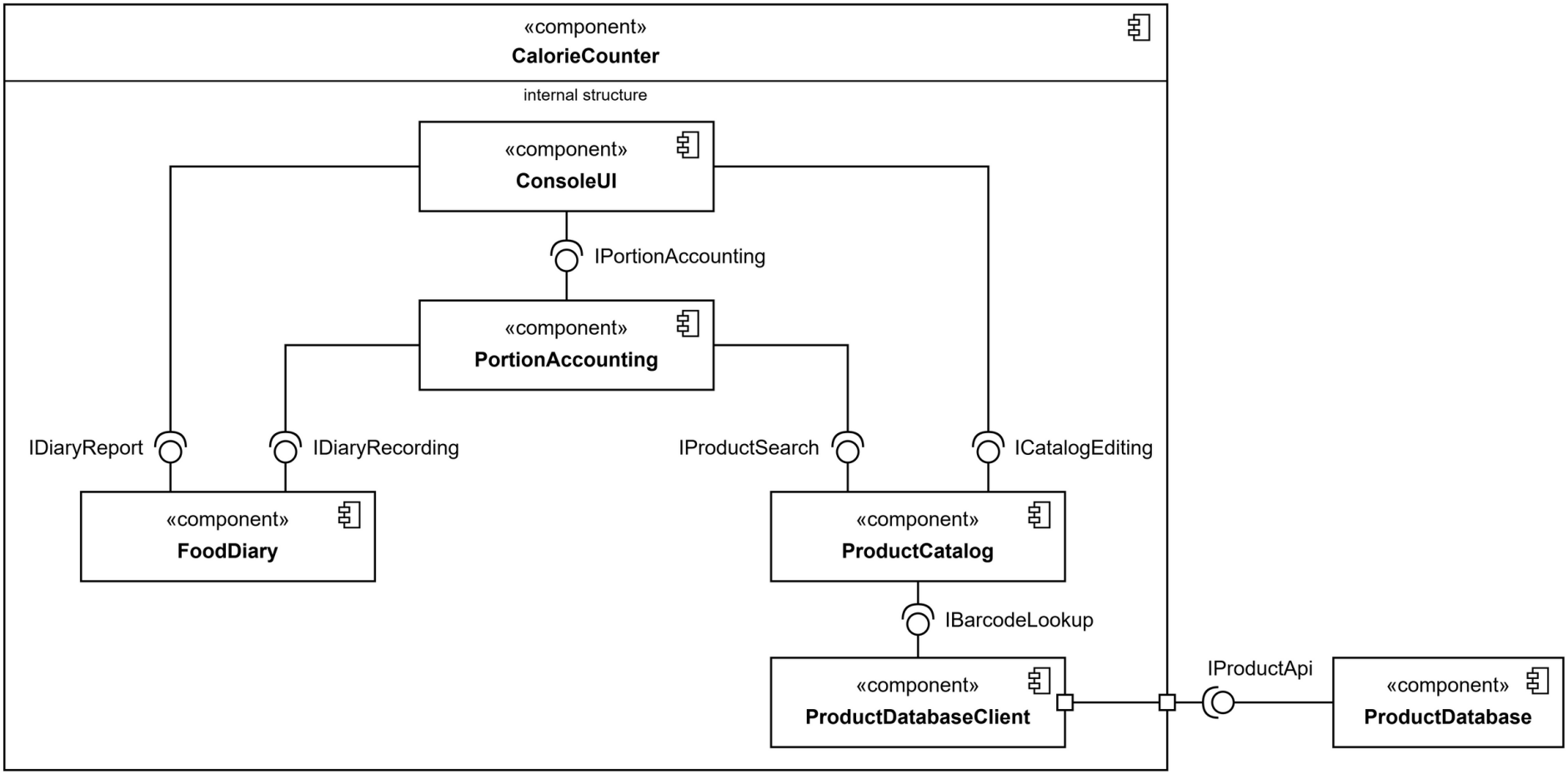
Таблица 3 – Описание диаграммы компонентов

Поле	**Описание**
Вопрос	Из каких заменяемых компонентов состоит программа и через какие интерфейсы они связаны между собой и с внешней базой продуктов?
Читатель	Тестировщик
Что сделает читатель	Проверит каждый компонент отдельно, подставив вместо соседних компонентов и внешней базы заглушки с теми же интерфейсами
Что показано	– пять компонентов программы внутри составного компонента CalorieCounter; – предоставляемые и требуемые интерфейсы и их соединения; – внешняя база продуктов, подключённая через порт и клиент; – отсутствие циклических зависимостей между компонентами.
Что не показано	– классы модели, которые передаются через интерфейсы; – операции интерфейсов — они перечислены в таблице 4; – формат запросов к внешней базе и поведение программы при её недоступности.
Допущения	– в коде лабораторной работы 2 интерфейсов нет: их операции — открытые операции классов PortionHandler, ProductCatalog и FoodDiary; – компонент ProductDatabaseClient и поиск по штрихкоду ещё не реализованы; – изменение и удаление продуктов и порций из лабораторной работы 3 добавят операции в существующие интерфейсы, но не изменят связей между компонентами.

Таблица 4 – Интерфейсы компонентов

Интерфейс	Предоставляет	Требует	Операции
IPortionAccounting	PortionAccounting	ConsoleUI	CreateBatch, Preview, Confirm
ICatalogEditing	ProductCatalog	ConsoleUI	AddProduct; добавление по штрихкоду не реализовано
IProductSearch	ProductCatalog	PortionAccounting	FindProduct
IDiaryRecording	FoodDiary	PortionAccounting	AddPortion
IDiaryReport	FoodDiary	ConsoleUI	CalculateDayKcal
IBarcodeLookup	ProductDatabaseClient	ProductCatalog	поиск продукта по штрихкоду, не реализован
IProductApi	ProductDatabase, внешняя система	ProductDatabaseClient через порт	запрос продукта по штрихкоду по протоколу HTTPS

Диаграмма компонентов счётчика калорий



Прямоугольник со значком — компонент; кружок — предоставляемый интерфейс, полукруг — требуемый

Кружок в полукруге — соединение компонентов; квадрат на границе — порт для внешней системы

Рисунок 2 – Диаграмма компонентов счётчика калорий

3 Диаграмма развёртывания

Диаграмма развёртывания показывает, на каких узлах размещаются файлы программы. Счётчик калорий — локальное приложение, поэтому все его файлы находятся на компьютере пользователя. Второй узел — сервер внешней базы продуктов, к которому программа обращается при поиске по штрихкоду.

Узел Workstation со стереотипом «device» — компьютер пользователя. Внутри него среда исполнения «executionEnvironment» .NET 10 Runtime: программа написана на C# для .NET 10 и без этой среды не запускается. В среде размещены два артефакта — файлы, которые получаются при сборке проекта. CalorieCounter.exe запускает программу, а CalorieCounter.dll содержит её код, то есть составной компонент CalorieCounter с рисунка 2. Артефакты обозначены значком листа в правом верхнем углу, их размещение показано вложением в узел.

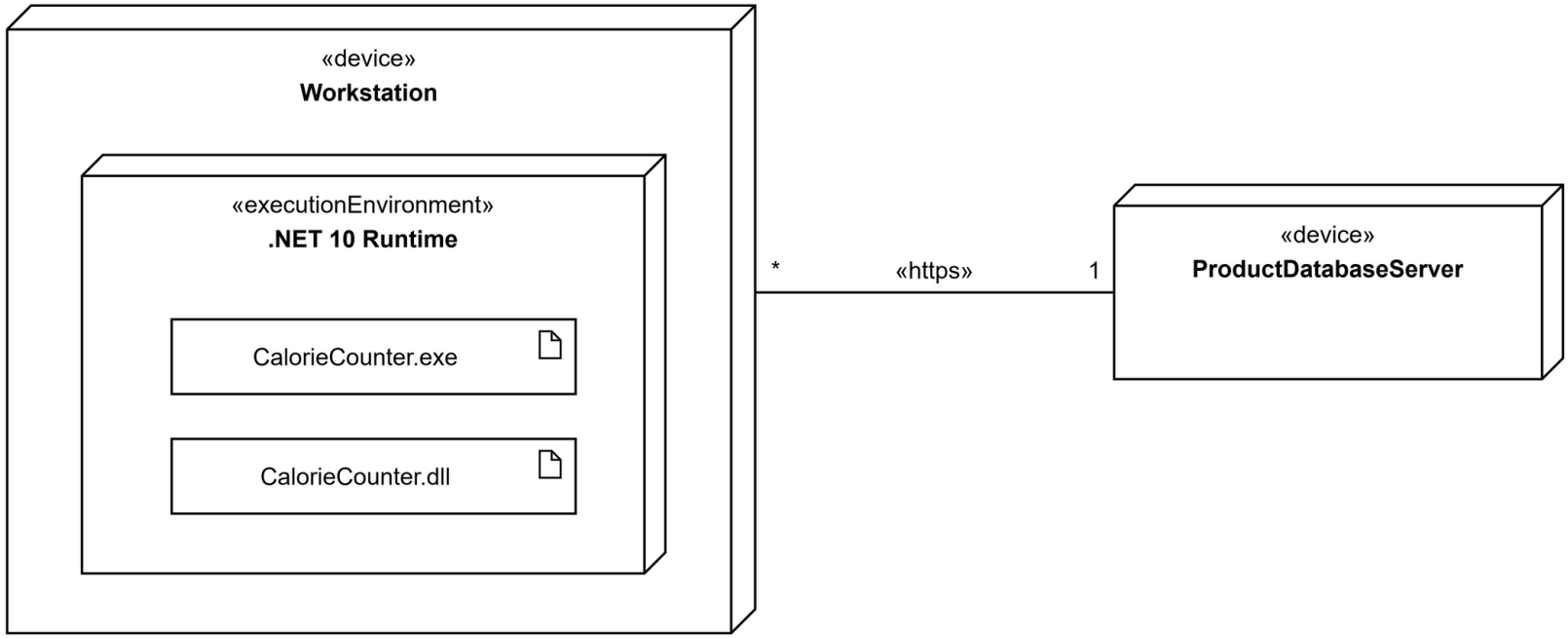
Узел ProductDatabaseServer — сервер внешней базы продуктов. Программа на нём ничего не размещает и его устройства не знает, поэтому содержимое сервера не показано. Узлы соединены линией связи, над ней указан протокол HTTPS. Кратность «*» у компьютера и «1» у сервера означает, что к одному серверу базы обращается много компьютеров пользователей.

Каталог и дневник хранятся в памяти программы и после её закрытия не сохраняются, поэтому файла данных на диаграмме нет. Диаграмма построена на уровне типов узлов, а не их экземпляров: она подходит для любого компьютера пользователя. Диаграмма представлена на рисунке 3, её описание — в таблице 5.

Таблица 5 – Описание диаграммы развёртывания

Поле	Описание
Вопрос	На каких устройствах и в какой среде работает счётчик калорий и как он связан с внешней базой продуктов?
Читатель	Администратор
Что сделает читатель	Установит на компьютер пользователя среду .NET 10 и файлы программы и проверит, что с компьютера открыт доступ к серверу базы продуктов по HTTPS
Что показано	– компьютер пользователя и сервер внешней базы продуктов — узлы-устройства; – среда исполнения .NET 10 на компьютере пользователя; – файлы программы, размещённые в этой среде; – связь компьютера с сервером по HTTPS и её кратности.
Что не показано	– операционная система компьютера и служебные файлы запуска .NET (CalorieCounter.runtimeconfig.json и CalorieCounter.deps.json); – устройство сервера внешней базы; – адрес сервера и формат запросов.
Допущения	– сервер базы продуктов — внешняя система, программа только обращается к нему; – связь с сервером нужна только для поиска по штрихкоду, остальные функции работают без неё; – данные хранятся в памяти программы и после её закрытия теряются, поэтому файла данных нет.

Диаграмма развёртывания счётчика калорий



Параллелепипед — узел: «device» — устройство, «executionEnvironment» — среда исполнения
Прямоугольник со значком листа — артефакт; линия между узлами — связь: протокол и кратности

Рисунок 3 – Сетевая архитектура счётчика калорий